

# A Representation Normalization Layer for Point Cloud Compression

September 18, 2026

## Abstract

Existing point cloud codecs are good at writing a given sequence of numbers compactly, but they never ask what those numbers represent. We place a representation normalization layer in front of the codec: the point cloud is returned to the quantities and the resolution that the acquisition device actually measures, and only then handed to an existing codec. The central observation is that, because the container decodes geometry first, the decoder holds every coordinate before it reads any attribute. Orders and predictors that are functions of the coordinates alone are therefore available at no cost in side information.

We evaluate on 15 files from 9 producers, covering airborne LiDAR, mobile LiDAR and triangulation scanners. One airborne LiDAR tile shrinks from 440 MB to 295.2 MB and remains bit-exact. In an end-to-end comparison that includes geometry we obtain a code length 6.3% below LASzip and 22.3% below the only G-PCC configuration that is lossless. Carrying every stream with the container’s own coder instead of an existing one, so that nothing external is inside, it is shorter than LASzip on all 15 LAS files by a median of 16.3%; measured on the geometry alone it is level with G-PCC, and the separation comes from the attribute side. We also quantify two negative results: below the point spacing an octree carries no extractable information, and differential coding of a repeat survey recovers only 1.3% of the geometry bits.

## 1 Introduction

Point cloud compression has developed around octree occupancy coding [1, 2]. MPEG’s G-PCC [3] is the culmination of that line, and in surveying LASzip [5] is the de facto standard. All of them solve the same problem: how to write a given sequence of coordinates and attributes as compactly as possible.

Open a file from production, however, and a different kind of waste appears. In one distributed airborne LiDAR product, two attributes were bit-for-bit identical. In one mobile LiDAR dataset the device measures range and angles, yet what is stored are single-precision Cartesian coordinates derived from them by trigonometry. The low mantissa bits produced by that conversion say nothing about the world, but they are required in order to reproduce the file exactly. In one shape database the point cloud was distributed as text.

None of this is a problem of the coding algorithm. It is a problem of representation. Existing codecs do not question the representation, so they encode the waste as given.

### 1.1 Contributions

1. We formalize the layer placed in front of the codec as a family of invertible maps, and by including the identity map in the family we guarantee constructively that the result is never worse than the original (§3).
2. We add to the family a scheme that codes normalized attributes in an order derived from the geometry. The coding scheme itself is the subject of the companion paper [7] (§4.3).

3. We quantify four directions that turn out to be closed (§7).

## 2 Related work

Octree coding begins with Meagher [1]; its first thorough application to point clouds is due to Schnabel and Klein [2]. G-PCC [3] codes geometry with an octree and attributes with the region-adaptive hierarchical transform (RAHT) [4] or with hierarchical neighbour prediction. The attribute coder we use as a candidate in our family belongs to the same framework as the latter, but differs in how the order is chosen and in how the colour transform is handled [7].

LASzip [5] compresses the point records of surveying LAS column by column, with a per-type predictor and arithmetic coding. Our container also uses LASzip inside.

Reversible luma–chroma transforms are known from Malvar et al. [6] as YCoCg-R and are standard in lossless still image coding. We apply this to point cloud attributes.

Ordering by space-filling curves goes back to Morton [8].

Prior work competes on writing the same representation more compactly. What we address is which representation to write in, and that is orthogonal to it.

### 3 Formulation

#### 3.1 Setting

We treat a point cloud as a sequence of geometry–attribute pairs

$$P = \{(\mathbf{x}_i, \mathbf{a}_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{Z}^3, \mathbf{a}_i \in \mathbb{Z}^M \quad (1)$$

The coordinates are already integers at the time of storage; scale and origin live in the header. The attributes are  $M$  integer fields, and  $a_i^{(j)}$  denotes the  $j$ -th component. We write  $\mathbf{f}^{(j)} = (a_1^{(j)}, \dots, a_N^{(j)})$  for the  $j$ -th field stacked as a column.

Let the reference codec be  $\mathcal{C} = (\mathcal{E}, \mathcal{D})$  and write its code length as  $\ell_{\mathcal{C}}(P) = |\mathcal{E}(P)|$ .

#### 3.2 The normalization layer

**Definition 1 (Normalization map)** A normalization map assigns to a point cloud  $P$  a transformed point cloud  $T(P)$  and side information  $s_T$ , such that an inverse  $T^{-1}$  exists with

$$T^{-1}(T(P), s_T) = P \quad (2)$$

When an error bound  $\epsilon$  is permitted, (2) is read instead as  $\|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2 \leq \epsilon \ (\forall i)$  with the attributes matching exactly.

The total code length after normalization is the sum of the code length of the transformed cloud and the length of the side information,

$$L(P; T) = \ell_{\mathcal{C}}(T(P)) + |s_T| \quad (3)$$

Counting the side information matters: without it, apparent gains are easy to produce on small files.

#### 3.3 The selection principle

Let  $\mathcal{T}$  be a finite family of candidate normalization maps, always containing the identity id. The map we adopt is

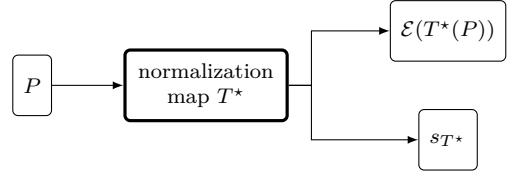
$$T^* = \arg \min_{T \in \mathcal{T} \cup \{\text{id}\}} L(P; T) \quad (4)$$

Because id belongs to the family, the following is immediate.

**Proposition 1 (No degradation)**  $L(P; T^*) \leq \ell_{\mathcal{C}}(P) + |s_{\emptyset}|$ , where  $s_{\emptyset}$  is the empty side information that records “no normalization applied”.

In our implementation  $|s_{\emptyset}| = 55$  bytes. Because of this guarantee the layer may be left enabled for every input.

Note that (4) demands measurement, not estimation. One could imagine deciding in advance whether a map will help, but the loss from a wrong decision is asymmetric. In our measurements, adopting a map correctly gained +13%, while adopting one wrongly cost −174%. As long as that asymmetry holds, the form of (4) is the only safe policy.



**Figure 1:** Where the layer sits. It does not replace the existing codec  $\mathcal{C}$ ; it precedes it. Decoding applies  $T^{*-1}$  after  $\mathcal{D}$ . That  $\mathcal{E}$  emits geometry in a form that can be decoded first is the premise of §4.3

### 4 The family of normalization maps

We describe the four kinds of map that make up  $\mathcal{T}$ .

#### 4.1 Algebraic relations between attributes

When field  $j$  is determined exactly from field  $k$  by integer arithmetic, the former need not be sent. That is, there exist integers  $(\alpha, \beta, \gamma)$  with  $\gamma \neq 0$  such that

$$a_i^{(j)} = \left\lfloor \frac{\alpha a_i^{(k)} + \beta}{\gamma} \right\rfloor \quad \forall i \in \{1, \dots, N\} \quad (5)$$

Here  $\alpha = \gamma = 1, \beta = 0$  is duplication and  $\alpha = 0$  is a constant. The search fixes  $(\alpha, \beta, \gamma)$  from two points and then verifies it at every point; a candidate that fails anywhere is rejected. Field names are never used.

When the fields do not agree exactly but differ by little, we add to the family a candidate that codes the residual  $\mathbf{f}^{(j)} - \mathbf{f}^{(k)}$  as a separate stream. Adoption is decided by (4).

#### 4.2 Returning to the acquisition representation

Let  $\mathbf{m} \in \mathcal{M}$  be the quantity the device actually measures and  $\Phi : \mathcal{M} \rightarrow \mathbb{R}^3$  the acquisition map that produces coordinates from it. The stored coordinates are

$$\mathbf{x}_i = Q_{\delta}(\Phi(\mathbf{m}_i)) \quad (6)$$

where  $Q_{\delta}$  is quantization with step  $\delta$ . If  $\Phi$  is known and invertible on its range, then instead of  $\mathbf{x}_i$  one may send  $Q_{\Delta}(\mathbf{m}_i)$ , quantized at the device’s own resolution  $\Delta$ . The reconstruction error is then bounded through the Jacobian  $J_{\Phi}$  of  $\Phi$  by

$$\|\Phi(Q_{\Delta}(\mathbf{m})) - \Phi(\mathbf{m})\|_2 \leq \frac{1}{2} \|J_{\Phi} \Delta\|_2 \quad (7)$$

For the spherical coordinates of a rotating LiDAR,  $\Phi(r, \theta, \phi) = (r \cos \theta \cos \phi, r \cos \theta \sin \phi, r \sin \theta)$ , this is

$$\|J_{\Phi} \Delta\|_2 = \sqrt{\Delta_r^2 + (r \Delta_{\theta})^2 + (r \cos \theta \Delta_{\phi})^2} \quad (8)$$

and substituting the maximum range  $r_{\max}$  gives the error bound.

The point is that if  $\Delta$  is the device resolution itself, then  $Q_{\Delta}(\mathbf{m})$  is informationally equivalent to the measurement. The bits created by  $\Phi$  and  $Q_{\delta}$  state nothing about the world.

Being informationally equivalent and being shorter to code are, however, different things. The substitution pays only when the components of  $\mathbf{m}$  form a sparse lattice, that is, when the number of distinct values of each component is far smaller than the number of points. In a rotating LiDAR the azimuth leads and the range follows: for each (elevation, azimuth) pair only the range varies, so the condition holds. In an indoor scan from a terrestrial laser scanner, by contrast, the range is nearly constant and the points cling to the walls, so in spherical coordinates all three components vary. Measured code lengths grew by 34–57% [7]. That the transform is known and invertible is a necessary condition only.

### 4.3 Coding the attributes

A normalized attribute either stays in the container or leaves it as a residual in an external stream. The coding scheme used in the latter case is not the subject of this paper; it is treated in the companion paper [7]. We state only the essentials.

The container of a point cloud decodes geometry first, so at the moment attributes are read the decoder already holds every coordinate. An order and a predictor that are functions of the coordinates alone can therefore be computed independently and identically by encoder and decoder, and need not be transmitted. The companion paper uses this freedom to predict from the preceding nearest neighbour in space-filling curve order, and applies a reversible luma–chroma transform to colour in addition. How many predictors to average is governed by the ratio of noise to smoothness in the attribute: attributes derived from orthoimagery do best with one, laser-derived attributes with five, and attributes without spatial structure do best with no differencing at all.

In the experiments of this paper we treat this scheme as one candidate in the family  $\mathcal{T}$ . Whether it is adopted is decided by (4).

### 4.4 Coder and container

Residual streams are coded with a deterministic, integer-only adaptive binary range coder. Because no floating point is involved, different implementations produce identical bit strings. The container also always records the side information  $s_T$ , that is the description of the applied map, and it is counted in (3).

## 5 Experiments

The data are listed in Table 1. Airborne LiDAR comes from AHN [9], mobile LiDAR from KITTI [10], and the triangulation scanners from the Stanford shape repository [11].

During the evaluation one file that we had originally included turned out to be an incomplete download truncated at exactly 10 MiB. Partial reads succeed on such a file, which makes it easy to miss. From then on we reconciled the header point count with the number of

**Table 1:** Evaluation data. For every file the point count in the header was reconciled with the number of points actually decoded

| Data                     | Producer / device | Points     |
|--------------------------|-------------------|------------|
| AHN4 31HZ1_20            | PDAL 2.3.0        | 49,292,437 |
| AHN4 31HZ1_21            | PDAL 2.3.0        | 47,488,087 |
| AHN5 31HZ1_20            | PDAL 2.7.1        | 29,464,008 |
| AHN3 31HZ1_20            | PDAL 2.3.0        | 20,132,124 |
| workshop_TM_551_101      | TerraScan         | 22,841,718 |
| autzen-2023.copc         | LasMonkey         | 21,233,219 |
| red-rocks                | PDAL 1.3.0        | 4,004,326  |
| autzen_trim              | PDAL 1.0.0        | 110,000    |
| simple1_4                | Global Mapper     | 22,600     |
| fullwave                 | YellowScan        | 10,750     |
| plane                    | PDAL 1.6.0        | 28,185     |
| vegetation               | RS Survey         | 10,683     |
| KITTI drive_0001         | Velodyne HDL-64E  | 13,178,808 |
| Bunny reconstruction     | Cyberware 3030MS  | 35,947     |
| Dragon reconstruction    | Cyberware 3030MS  | 437,645    |
| Armadillo reconstruction | Cyberware 3030MS  | 172,974    |

points actually decoded for every file, and excluded the file in question.

All measurements are end-to-end coding. The baseline is the original file rewritten once, and its code length is  $\ell_C(P)$ ; we verified that this baseline reproduces the distributed file to within 0.03%. For measurements on LAS/LAZ the inner codec  $\mathcal{C}$  is LASzip [5]. For lossless rows we check on every run that all restored fields match the original exactly; for rows with an error bound we check that every point respects the declared value of (7). No verification is done by sampling. The measurement machine has 16 CPU cores; no GPU and no learned model are used.

## 6 Results

### 6.1 End to end

Table 2 gives the results. One airborne LiDAR tile goes from 439,622,602 bytes to 314,542,058 bytes (−28.5%), with geometry and attributes both restored bit-exactly. It takes 445 seconds and 22.8 GB of peak memory.

The inner codec  $\mathcal{C}$  is LASzip [5] in all three columns. The middle and right columns both place our method in front of it and differ only in how the attribute predictor is taken: the middle uses the immediately preceding point in storage order, the right uses the preceding nearest neighbour in an order derived from the geometry [7].

Apart from AHN4, the files at the top of the table carry almost no algebraic relation (5) between attributes. For those files the earlier construction reduced the size by only 0.0–0.8%. Adding §4.3 moves the basis of the reduction from redundancy left behind by the producer to the mismatch between the storage representation of an attribute and its acquisition representation. The latter is a far more general property.

For the two files in the lower block, (4) chose id. Both are small files in which the gain falls below the fixed cost of the side information, so the map was not adopted and the result lands on the baseline plus  $|s_0|$ .

**Table 2:** Measured on all points, in bits per point. The inner codec  $\mathcal{C}$  is LASzip in all three columns, and our method precedes it. The middle and right columns differ only in how the attribute predictor is taken. Bold marks values below LASzip alone

| File                | Producer      | Points [ $10^4$ ] | LASzip alone | LASzip + ours |                | Reduction |
|---------------------|---------------|-------------------|--------------|---------------|----------------|-----------|
|                     |               |                   |              | storage order | geometry order |           |
| AHN4 31HZ1_20       | PDAL 2.3.0    | 4,929.2           | 71.349       | 54.349        | <b>47.908</b>  | −32.9%    |
| fullwave            | YellowScan    | 1.1               | 73.480       | 73.794        | <b>60.871</b>  | −17.2%    |
| AHN5 31HZ1_20       | PDAL 2.7.1    | 2,946.4           | 57.821       | 57.363        | <b>51.396</b>  | −11.1%    |
| plane               | PDAL 1.6.0    | 2.8               | 16.689       | 16.883        | <b>14.976</b>  | −10.3%    |
| AHN3 31HZ1_20       | PDAL 2.3.0    | 2,013.2           | 50.059       | 49.810        | <b>46.737</b>  | −6.6%     |
| autzen-2023.copc    | LasMonkey     | 2,123.3           | 67.049       | 67.049        | <b>62.620</b>  | −6.6%     |
| workshop_TM_551_101 | TerraScan     | 2,284.2           | 37.292       | 37.292        | <b>35.359</b>  | −5.2%     |
| red-rocks           | PDAL 1.3.0    | 400.4             | 20.354       | 19.730        | <b>19.525</b>  | −4.1%     |
| autzen_trim         | PDAL 1.0.0    | 11.0              | 43.507       | 43.525        | <b>41.729</b>  | −4.1%     |
| simple1_4           | Global Mapper | 2.3               | 17.661       | 17.680        | 17.680         | +0.1%     |
| vegetation          | RS Survey     | 1.1               | 49.845       | 50.165        | 49.886         | +0.1%     |

**Table 3:** End-to-end comparison including geometry. Airborne LiDAR; geometry and the three colour components are lossless and reconciled over all points. Bits per point. All values are measured

| Method             | Order   | 2M points |             |              | 8M           |
|--------------------|---------|-----------|-------------|--------------|--------------|
|                    |         | Geom.     | Col.        | Total        | Total        |
| G-PCC predicting   | geom.   | 25.36     | 12.78       | 38.14        | 38.57        |
| G-PCC geom. + ours | geom.   | 25.36     | 7.43        | 32.89        | 33.31        |
| LASzip alone       | acquis. | 22.22     | 9.42        | 31.64        | 32.48        |
| LASzip + ours      | acquis. | 22.22     | <b>7.43</b> | <b>29.65</b> | <b>30.22</b> |

This is exactly the behaviour Proposition 1 describes, and it is why the layer may be enabled regardless of the input.

Note that **plane** is a file our earlier implementation judged to contain “no normalizable redundancy”. Replacing the evaluation in (4) by measurement rather than a proxy changed it to −10.3%. The judgement had been wrong; the data were not free of redundancy [7].

## 6.2 Comparison with other codecs

### Levelling the ground

Before comparing we level the content. The attributes G-PCC [3] can carry are limited to colour and reflectance; it cannot accept the 12 fields the airborne LiDAR actually has, and those fields occupy 55.5% of the original file. The comparison is therefore restricted to geometry and the three colour components.

We do not level the point order. The reason is given below (§6.2).

### Results

Table 3 gives the comparison. Our method is 6.3% smaller than LASzip [5] and 22.3% smaller than the lossless G-PCC configuration. At 8M points the figures are 7.0% and 21.6%; quadrupling the scale does not change the ranking.

Broken down, on colour our method is 21.1% smaller than LASzip and 41.8% smaller than G-PCC. On geometry LASzip is 12.4% smaller than G-PCC against an untuned TMC13 configuration; against a tuned one (24.492) the margin is 9.3%, and against the best we found (24.375, IDCM=2) it is 8.8%.

### Order determines the code length

Which geometry coder wins reverses with the point order. The LASzip geometry predictor is a sequential predictor along the scan line, and acquisition order is the input it handles best. Reordering the same point set into a geometry-derived order raises the code length from 22.219 to 28.232 bits per point, a 27.1% increase, which puts it below the 25.363 of G-PCC. Without reordering it is above G-PCC [7].

For this reason the input must not be uniformly reordered before a comparison. Reordering destroys the premise of one of the coders; it is not a levelling operation. That G-PCC does not preserve order should be treated as a limitation on the G-PCC side wherever order is needed.

### Combining the two is a loss

We implemented and measured a construction in which G-PCC handles the geometry and our method handles the attributes. The decoder must be able to build a canonical order from the coordinates, so both sides compute Morton order independently and the attributes are coded in that order. We verified that the coordinate sequences the two orders produce agree exactly (at 8M points, zero displaced coordinates and two points with fully duplicate coordinates). Since the arrangement of duplicate points is not determined by geometry, the reconciliation is done on the multiset of coordinate–attribute pairs.

Restricted to colour, the result is 32.889 bits per point at 2M and 33.307 at 8M. Both were restored and reconciled over all points. As Table 3 shows, these are 10.9% and 10.2% larger than the construction with

LASzip inside (29.653 and 30.224).

Passing all 18 fields of the original data widens the gap further: 72.410 against 52.773 at 2M points and 73.979 against 53.467 at 8M, an increase of about 38% in both cases.

The reason is the effect of reordering, measured in the companion paper [7]: G-PCC forces every point into a single geometry-derived order, which destroys the structure of attributes that depend on acquisition order. Our method keeps the container in acquisition order and treats only the spatial-prediction residual stream in a geometry-derived order. Being able to use two orders at once is the advantage of this construction.

### 6.3 The container standing alone

Every configuration reported so far puts an existing codec inside the container. The coder of §4.4 can carry every stream itself, and we report that configuration here because it is the one that owes nothing to an external implementation. The candidate family of §3.3 is unchanged; only the set of coders the selection may choose from is now our own.

The corpus is the 15 LAS files we hold: the twelve LAS entries of Table 1 together with two USGS tiles and one file carrying extra bytes. The four non-LAS sources are excluded because the baseline has to be written in a LAS container. The baseline is LASzip over the columns the container reads, so both sides code the same values.

Table 4 gives the result. The container is shorter than LASzip on all 15 files, by a median of 16.3% (range 9.1–37.6%; the median sits on red-rocks, and one file moving past the middle can shift it by more than a point, so the per-file rows are the ones to read). The same measurement at  $2 \times 10^5$  points with an earlier build gives 12.4%, so the figure is not an artefact of the extract size.

The two sides of the container do not contribute equally. Splitting the code length at the boundary between coordinates and attributes, the geometry side is shorter on all 15 files with a median of −13.1%, the attribute side on all 15 with a median of −18.1%. Measured on geometry alone the container is level with G-PCC over 17 files, the twelve above together with five further sources including a mobile LiDAR drive and a terrestrial scan — the median of the per-file ratio is 0.91, with quartiles 0.87 and 1.09, shorter on 11 of the 17, while against LASzip on the same geometry it is shorter on all 17 (median 0.89) — and no ordering of the three coders follows from that side. The separation is on the attribute side, and §4.3 is what produces it.

#### What it costs

Against LASzip the container is  $44.8\times$  slower to encode and  $10.3\times$  slower to decode (medians over the 15 files;  $27.6\times$  and  $4.0\times$  at  $2 \times 10^5$  points with an earlier build, so the decode gap widens with scale). Encoding is the expensive direction because the selection principle codes every candidate and keeps the shortest, while decoding passes through the one that was kept. Against G-PCC

on geometry alone the container is now slower ( $1.66\times$ , an axis on which repeated runs of the same binary move by 10%): the candidate family has grown, and carrying every dimension of the source added columns to code.

Memory is the axis on which it loses to both. At  $2 \times 10^6$  points the peak resident set is a median of 728 bytes per point over all columns. On geometry alone, where the three coders can be put side by side, the median of the per-file ratio is  $4.4\times$  against G-PCC and  $9.8\times$  against LASzip. The octree slices its input and its working set is bounded; ours holds the whole point set and every candidate under trial, and grows linearly. What it holds is already narrow: each column is stored at the width that column needs rather than at 64 bits, a constant column costs eight bytes in total, and the residual of a candidate is held once and in place. What remains is the selection principle itself — every candidate under trial needs its own working array — and the neighbour tables for all three predictor counts. Both are what buys the code length.

### 6.4 Returning to the acquisition representation

Over 108 frames of mobile LiDAR (13.18M points), against 96.000 bits per point for uncompressed single-precision Cartesian coordinates, lossless compression gives 46.183 and the spherical quantization based on (8) ( $\Delta_r = 2\text{ mm}$ ,  $\Delta_\theta = \Delta_\phi = 0.002^\circ$ ) gives 14.184 bits per point, a reduction of 85.2%. The measured maximum reconstruction error is 2.11 mm, which does not degrade the accuracy the device guarantees: its range resolution is 2 mm and its ranging accuracy  $\pm 2\text{ cm}$ . All 108 frames fall within 14.065–14.377.

For the three triangulation-scanner models, against 675.0/618.4/320.0 bits per point in the distributed format, a lattice at one eighth of the point spacing gives 9.55/10.76/6.50 bits per point (−98.0–98.6%). Most of that reduction comes from the distributed format being text, and should not be reported as an achievement of coding technology.

### 6.5 Moving the accuracy requirement

Table 5 is not a table of compression ratios but a table of requirements. Each row is the consequence of a decision about what accuracy to demand, and the numbers change when the decision changes. Every error bound was measured over all 49.29M points and respects its declared value.

## 7 Quantifying the limits

A negative result is worth as much as a positive one when the question is where to spend effort.

### 7.1 The information limit of the octree

Let  $c_n \in \{1, \dots, 255\}$  be the occupancy byte of node  $n$  at depth  $\ell$  and  $\hat{c}_n$  the context available from its parent.

**Table 4:** The container carrying every stream with its own coder, with no existing codec inside. The 15 LAS files of the corpus, the centre  $2 \times 10^6$  points of each where the file holds that many and the whole file otherwise. Every row is lossless and was decoded and compared column by column. Bits per point; the baseline is LASzip over the same columns. The last two columns split the reduction by the two sides of the container

| Data          | All columns |                | Red.           | Reduction by side |              |
|---------------|-------------|----------------|----------------|-------------------|--------------|
|               | LASzip      | Ours           |                | Geometry          | Attributes   |
| AHN4 31HZ1_20 | 69.817      | <b>43.583</b>  | −37.6%         | −17.9%            | −46.3%       |
| extra bytes   | 218.471     | <b>136.841</b> | −37.4%         | −4.3%             | −46.7%       |
| plane         | 16.844      | <b>11.094</b>  | −34.1%         | −18.1%            | −44.6%       |
| AHN4 31HZ1_21 | 77.401      | <b>51.680</b>  | −33.2%         | −19.5%            | −41.7%       |
| simple1_4     | 17.730      | <b>12.020</b>  | −32.2%         | −49.4%            | −20.0%       |
| fullwave      | 149.503     | <b>119.615</b> | −20.0%         | −16.2%            | −20.9%       |
| vegetation    | 49.845      | <b>40.814</b>  | −18.1%         | −5.2%             | −30.6%       |
| red-rocks     | 20.749      | <b>17.361</b>  | −16.3%         | −26.3%            | −10.0%       |
| workshop      | 37.091      | <b>31.628</b>  | −14.7%         | −16.7%            | −11.5%       |
| autzen_trim   | 43.880      | <b>37.533</b>  | −14.5%         | −10.5%            | −18.1%       |
| AHN5 31HZ1_20 | 59.091      | <b>51.744</b>  | −12.4%         | −8.7%             | −15.4%       |
| autzen-2023   | 66.665      | <b>58.769</b>  | −11.8%         | −10.8%            | −12.5%       |
| USGS NY       | 38.710      | <b>34.846</b>  | −10.0%         | −13.1%            | −1.2%        |
| AHN3 31HZ1_20 | 53.702      | <b>48.442</b>  | −9.8%          | −8.5%             | −10.9%       |
| USGS AK       | 35.629      | <b>32.380</b>  | −9.1%          | −9.1%             | −9.1%        |
| median        |             |                | − <b>16.3%</b> | −13.1%            | −18.1%       |
| shorter on    |             |                | <b>15/15</b>   | <b>15/15</b>      | <b>15/15</b> |

**Table 5:** A whole airborne LiDAR tile under different accuracy requirements (measured without §4.3)

| Requirement   | Error bound | bits/point | Reduction |
|---------------|-------------|------------|-----------|
| Lossless      | —           | 54.349     | −23.8%    |
| 8 mm lattice  | 6.93 mm     | 46.243     | −35.2%    |
| 16 mm lattice | 13.86 mm    | 43.698     | −38.8%    |
| 32 mm lattice | 27.71 mm    | 41.506     | −41.8%    |

The information the context provides is the mutual information

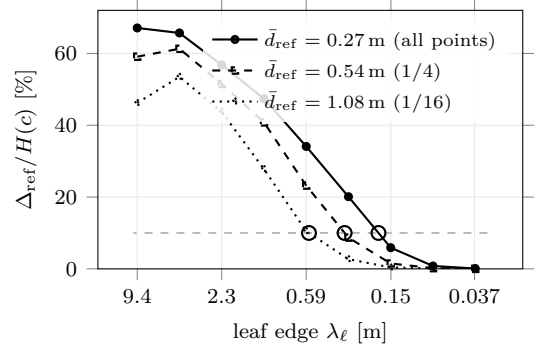
$$I(c; \hat{c}) = H(c) - H(c | \hat{c}) \quad (9)$$

Write  $\lambda_\ell$  for the leaf edge length and  $\bar{d}$  for the median nearest neighbour distance.

**Proposition 2** *If  $\lambda_\ell$  is small enough relative to  $\bar{d}$  that each occupied node contains at most one point, then  $c_n$  points to a single child, so  $c_n = 2^{k_n}$  with  $k_n \in \{0, \dots, 7\}$  and  $H(c) = H(k) \leq 3$  bits. Equality holds when  $k$  is uniform.*

In our measurements, in the region where  $\lambda_\ell$  falls below one quarter to one eighth of  $\bar{d}$ , all 13 datasets show  $H(c) = 2.976$ – $3.000$  and sit on the equality side of Proposition 2. This means that no context exists: almost every neighbouring cell is empty, so there is nothing left to condition on.

Summing the roughly seven levels below the point spacing yields about 1 bit per point, which is 4–5% of the 22 bits per point spent on geometry.



**Figure 2:** Behaviour of (10) as the reference cloud is thinned. The target is the same in all three cases, with its point spacing fixed at 0.085 m, so the movement of the knee comes only from the reference spacing  $\bar{d}_{\text{ref}}$ . The dashed line is the 10% threshold that defines  $\lambda^*$ , and the circles are the interpolated intersections, that is the  $\lambda^*$  of (11): 0.179, 0.312 and 0.560 m from the left

## 7.2 Differential coding of a repeat survey

Given two point clouds of the same area measured at different times, we asked whether one can be sent cheaply using the other as a reference. Descending the octree of the target, we add the occupancy byte  $c_n^{\text{ref}}$  of the reference at the same node to the context. The decoder holds the reference too, so this involves no transmission. The gain is measured by

$$\Delta_{\text{ref}}(\lambda_\ell) = H(c) - H(c | c^{\text{ref}}) \quad (10)$$

Descending to the actual resolution of the data ( $\lambda_\ell = 1.1$  mm), the reduction over all geometry bits was 1.3%. As a control, replacing the reference by a different location with the same point density and the same kind

of terrain drops the reduction from 2.6% to 0.2%, so being the same location is genuinely doing work.

The effect is nevertheless concentrated at coarse lattices. Figure 2 shows (10) as the reference is thinned to 1/4 and 1/16. Although the point spacing of the target is held fixed, the knee moves, and the leaf size  $\lambda^*$  at which  $\Delta_{\text{ref}}$  falls below 10% follows

$$\lambda^* \approx \alpha \bar{d}_{\text{ref}}, \quad \alpha \in [0.5, 0.7] \quad (11)$$

(quadrupling  $\bar{d}_{\text{ref}}$  moves  $\lambda^*$  by a factor of 3.1). Since the reference has no surface finer than its own point spacing, it is only to be expected that it cannot predict scales below that.

What two surveys share is the shape of the terrain and the buildings, not where the rays happened to land. And 78–89% of the geometry bits are spent on the latter.

### 7.3 Other directions

Summarizing shape by the Euler characteristic transform works, but for the same purpose it loses to the D2 shape distribution by a factor of 380 in time. Determining the accuracy requirement from persistent homology [12] was never the binding constraint in any of the 13 datasets: the lower bound imposed by the device resolution was always 10–40 times tighter, and we could not find a case in which topology mattered.

## 8 Discussion

### 8.1 Where the bits are

The 54.349 bits per point after normalization for airborne LiDAR divide into 41.1% geometry and 58.9% attributes. Research in point cloud compression concentrates on geometry, yet six tenths of the bits in real data belong to the attributes. We ourselves first pursued four directions on the geometry side, and only after all of them closed did we notice that we had not looked at the larger half.

Furthermore, of the 22.2 bits per point of geometry, we estimate that 11–22% describes the shape of the surface. That value depends on our definition of “surface” as a lattice at the point spacing. The rest is where the rays landed, which states nothing about the world.

### 8.2 Attributes divide by nature

Comparing prediction in a geometry-derived order against prediction in storage order separates the attributes cleanly. Attributes derived from orthoimagery (three colour components and near infrared) are spatially smooth and improve by 43.6–43.7% with a single nearest neighbour. Device-derived attributes (amplitude, reflectance) are noisy and benefit from averaging several. The waveform deviation has structure neither in space nor in acquisition order, and is best sent without differencing. The companion paper [7] formalizes this distinction as a condition on the ratio of noise to smoothness.

Seen through the framework of this paper, the distinction is between attributes that describe a property of a place and attributes that are a record of an individual shot. The former are an orthoimage resampled onto points, that is, a two-dimensional field by origin. What is really an image is being stored as though it were a per-point measurement.

### 8.3 Order is part of the representation

The arrangement of the points by itself determines the code length. For the same point set, placing it in acquisition order or in a geometry-derived order moves the geometry code length by 27%, and moves acquisition-order-dependent attributes such as the timestamp by more than a factor of five [7].

This can be read as an instance of the theme of this paper. The arrangement of points is part of the representation just as coordinates and attributes are, and which arrangement to write in is part of the question of which representation to write in. Existing codecs do not question the arrangement they are given, so a careless reordering silently degrades the code length.

### 8.4 Where the reduction comes from

Splitting the reduction by means, the contribution of the coding algorithm itself is less than half of the total. The rest comes from correcting the container, from returning to the acquisition representation, and from agreeing on an accuracy requirement. The last of these is not compression but a decision about requirements, and should be reported separately.

### 8.5 Limitations

First, evaluating (4) by measurement is essential. The decrease in  $\ell_C$  when a field is removed from the container does not equal the standalone cost of that field, because existing codecs use context across columns. Our early implementation used the standalone cost as a proxy when narrowing down candidates, and this produced a wrong decision worth 2.3 bits per point on one field. We now write the container without the attribute and measure the marginal cost, and because marginal costs are not additive we also run a greedy verification afterwards. Replacing estimation by measurement recovered 3.1 bits per point in total [7]. The greedy verification is, however, a one-at-a-time check and does not exhaust combinations of candidates.

Second, the effect of §4.3 cannot be measured on a subsample. The distance to a predictor depends on how densely the points are packed, and since storage is in spatial order, an initial prefix corresponds to a long thin region. Measured deviations went both ways (2.42% against 6.07%, and 6.37% against 4.70%). Every table in this paper was remeasured over all points.

Third, the acquisition modalities we evaluated are four: airborne LiDAR, mobile LiDAR, terrestrial laser scanning and triangulation scanners. For terrestrial scanning we confirmed that §4.2 is counterproductive. Mobile mapping, underwater acoustic sounding and

interferometric synthetic aperture radar are untested. The mobile LiDAR result also rests on a single drive with a single device. We could not obtain public terrestrial data carrying attributes, so the generality of the attribute side is unconfirmed outside airborne LiDAR.

Fourth, computation. The whole tile takes 445 seconds, of which 226 seconds are spent constructing the family  $\mathcal{T}$ , that is, searching for relations between pairs of fields. That is an  $O(M^2N)$  algorithm which should be improvable by narrowing the candidates on a sample, but we have not implemented it.

Fifth, the container of §6.3 now reads every field a LAS file can hold, and this changed the measurement. An earlier version packed only the return number and the number of returns; the scan direction flag, the edge-of-flight-line flag, the scanner channel, the classification flags, the waveform fields of point formats 4, 5, 9 and 10, the extra-bytes array types and the variable-length records were dropped, and on 13 of the 15 files at least one dimension of the original did not come back. Carrying them costs code length — the all-column median moved from  $-14.0\%$  to  $-11.5\%$  when they were first added, and the predictor and model work of §4.4 brought it back and past, to  $-14.3\%$ , and a last round (signed and return-type contexts for the attribute residuals, a binary model whose adaptation rate depends on how often its context has been seen, and signed contexts for the remaining geometry coders, and return-type contexts for the geometry coders that send residuals in point order) to the  $-16.3\%$  of Table 4. Two late steps used structure outside the coordinate values. The returns of one pulse lie on one ray; coding the later returns of a pulse under their own context, and keeping them out of the history a trend-adding predictor extrapolates from, moved six of the fifteen files. Using the ray itself as a predictor did not help. Predicting  $z$  from already-decoded points near the current  $(x, y)$ , switched against the current predictor by recent error, moved one file by  $0.07\%$  and costs  $19\%$  of encoding time. Every number in this paper is measured with them carried. What is verified is: round-trip equality on all columns for 15 of 15 files, byte-identical output across 1, 3, 7 and 16 threads for 17 of 17 inputs, and a field-by-field comparison of the rewritten LAS — dimensions, variable- and extended-variable-length records and header fields — matching on all 15 files (the records of the COPC file are compared as raw bytes, which the earlier tool could not do). The container is therefore a replacement for the distribution format on this corpus, not only a coder.

Sixth, geometry may also be coded with a declared error bound rather than exactly. This is off unless asked for. On the mobile LiDAR drive, quantising to a polar lattice gives a median of 17.95 bpp at a measured bound of 2 mm and 14.23 bpp at 5 mm, against 21.62 bpp exact —  $-17.0\%$  and  $-34.2\%$ . The declared bound is enforced end to end: the encoder measures the error through the same rounding the decoder performs, including the final write back to the source representation, and tightens the lattice until the measured maximum is inside the bound. Attributes stay bit-exact in this mode.

Seventh, the working set of that container grows linearly with the point count while an octree’s does not. Measured on geometry alone with a single pinned candidate it is nevertheless below the octree at every size we tried, from 30 MB at  $5 \times 10^5$  points to 190 MB at  $4 \times 10^6$ , against 102 and 265 MB; it is the candidate trials of the full container that put it above. We record that this was the axis we watched least: during the work reported here the working set grew by a factor of 4.5 without the code length changing, and was found only when we went back to reproduce the figures of the companion paper on input order.

## 9 Conclusion

We proposed placing a normalization layer in front of the coder, returning the point cloud to the quantities and the resolution of the acquisition device before handing it to an existing codec. By formalizing the layer as a family of invertible maps, including the identity in the family, and selecting by measured code length, we guaranteed constructively that the result never exceeds the original code length.

The property at the centre is that the container decodes geometry first. Any order and any predictor that is a function of the coordinates alone is therefore available without side information. Attribute coding built on that freedom gave a code length  $21.1\%$  below LASzip and  $41.8\%$  below the lossless G-PCC configuration under identical conditions. End to end including geometry it is  $6.3\%$  below LASzip and  $22.3\%$  below G-PCC. One airborne LiDAR tile shrinks by  $28.5\%$  while remaining lossless. Mobile LiDAR shrinks by  $83.1\%$  losslessly against the four `float32` values per point that the source stores ( $x$ ,  $y$ ,  $z$  and intensity, 128 bits), and by  $88.9\%$  at a measured error bound of 5 mm — a quarter of the device’s  $\pm 2$  cm ranging accuracy — with the attributes still bit-exact. Both are medians over the 20 frames of the sweep; over all 108 frames the lossless figure is  $83.2\%$ .

We also showed quantitatively that an octree extracts no information below the point spacing, and that differential coding of a repeat survey recovers only  $1.3\%$  of the geometry bits.

Six tenths of the bits in real data are in the attributes, and eight tenths of the geometry bits describe where the rays landed. We believe the next headroom in point cloud compression lies not in a better entropy coder but in redefining what should be preserved.

## References

- [1] D. Meagher, “Geometric modeling using octree encoding,” *Computer Graphics and Image Processing*, vol. 19, no. 2, pp. 129–147, 1982.
- [2] R. Schnabel and R. Klein, “Octree-based point-cloud compression,” *Symposium on Point-Based Graphics*, pp. 111–120, 2006.
- [3] MPEG 3DG, *G-PCC Codec Description*, ISO/IEC JTC1/SC29/WG7, 2023. Reference implementation mpeg-pcc-tmc13 v23.0.
- [4] R. L. de Queiroz and P. A. Chou, “Compression of 3D point clouds using a region-adaptive hierarchical transform,” *IEEE Trans. Image Processing*, vol. 25, no. 8, pp. 3947–3956, 2016.

- [5] M. Isenburg, “LASzip: Lossless compression of LiDAR data,” *Photogrammetric Engineering & Remote Sensing*, vol. 79, no. 2, pp. 209–217, 2013.
- [6] H. S. Malvar, G. J. Sullivan, and S. Srinivasan, “Lifting-based reversible color transformations for image compression,” *Proc. SPIE*, vol. 7073, 2008.
- [7] Companion paper, “Lossless coding of airborne LiDAR attributes in a geometry-derived order.” Covers the attribute coding scheme, the condition on the number of predictors, the measured verification of the selection, and the limits on the geometry side.
- [8] G. M. Morton, *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*, IBM Ltd., Ottawa, 1966.
- [9] Actueel Hoogtebestand Nederland (AHN3 / AHN4 / AHN5), [geotiles.citg.tudelft.nl](http://geotiles.citg.tudelft.nl).
- [10] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The KITTI dataset,” *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [11] Stanford Computer Graphics Laboratory, *The Stanford 3D Scanning Repository*.
- [12] The GUDHI Project, *GUDHI User and Reference Manual*, 3.13, 2025.